Chapter Five- DATA WRANGLING COMBINING AND MERGING DATA SETS

OBJECTIVES

The main goal of this chapter is to help students learn, understand and practice the data science approaches, which include the study of latest data science tools with latest programming languages. The main objectives of this chapter are data wrangling which includes data discovery, structuring, cleaning, enriching, validating and publishing, combining and merging datasets, data reshaping, pivoting, transformation, string manipulation operations and regular expression.

5.1 INTRODUCTION

How a Data Scientist works:

- Ask the right questions - To understand the business problem.
- Explore and collect data - From database, web logs, customer feedback, etc.
- Extract the data - Transform the data to a standardized format.
- Clean the data - Remove erroneous values from the data.
- Find and replace missing values - Check for missing values and replace them with a suitable value (e.g. an average value).
- Normalize data - Scale the values in a practical range (e.g. 140 cm is smaller than 1,8 m. However, the number 140 is larger than 1,8. - so scaling is important).
- Analyze data, find patterns and make future predictions.
- Represent the result - Present the result with useful insights in a way the "company" can understand.

5.2 DATA WRANGLING

Data wrangling is a key component of any data science project. Data wrangling is a process where one transforms "raw" data for making it more suitable for analysis and it will improve the quality of the data.

Data wrangling is the process of collecting, gathering and transforming of raw data into appropriate format for accessing, analyzing, easy understanding and further processing for better and quick decision making.

It is also known as **Data Munging** or **Data Pre-Processing**.

Data wrangling is a crucial first steps in the preparation of data for broad analysis of huge amount of data or big data. It requires significant amount of time and efforts. If data wrangling is properly conducted, it gives you insights into the nature of the data. It is not just a single time process but it is an iterative or repetitive process. Each step in the data wrangling process exposes the new potential ways for the data re-wrangling towards deriving the goal of complex data manipulation and analysis.

5.2.1 Steps for Data Wrangling

Data wrangling process includes **six** core activities:

- Discovery
- Structuring
- Cleaning
- Enriching
- Validating

- Publishing

- **Discovery**: Data discovering is an umbrella term. It describes the process to understand the dataset and insight into it. It involves the collection and evaluation of data from various sources and is often used to identify the spot trends and detecting the patterns of the data and gain instant insight.

Now a days large business organization or companies have a large amount of data related to the customers, suppliers, production, sells, marketing etc. For example, if a company have a customer database, you can identify that the most of the customers are from which part of the city or state or country.

- Structuring: Data is coming from the various sources with the difference formats. Structuring is necessary because of the different size and shape of the data. Data structuring is the process or actions that change the form or schema of the dataset. Data splitting into the columns, deleting some fields from the dataset, pivoting rows are the form of data structuring.

- **Cleaning**: Before start the data manipulation or data analysis, you need to perform the data cleaning. Data cleaning is the process to identify the data quality related issues, such as missing or mismatched values, duplicate records etc. and apply the appropriate transformation to correct, replace or delete those values or records from the dataset to make high quality of data.

- **Enriching**: The data is useful for decision making process in the business. The data needed to take business related decision can be stored into multiple files.

To gather all necessary insights into single file and you need to enriched your existing dataset by performing joining and aggregating multiple data sources.

- **Validating**: After completion of data cleaning and data enriching, you need to check the accuracy of the data. If dataset is not accurate, it might be creating a problem. It is necessary to do the data validation. This is the final check that any missing or mismatched data was not corrected during the transformation process. It is also need to validate that output dataset has the intended structure and content before publishing it.

- **Publishing**: After successful completion of data discovering, structuring, cleaning, enriching and validating, it's a time to published the wrangled output data for the further analytics processes, if any. The published data can be uploaded in the 55 organization‟s software or store into the file in a specific location where organizations peoples knows it is ready to use.

5.2.2 Tools for Data Wrangling

There are some tools available for the data wrangling. Some of the popular tools are as follow:

Python

R

Tabula: Tabula is a tool for liberating data tables trapped inside the PDF files. It allows the user to upload the files in PDF format and extract the selected rows and columns

from any tables available in the PDF file. It supports to extract this data from PDF to CSV or Microsoft Excel file format.

Data Wrangler:

It is an interactive tool for data cleaning. It takes the read word data and transform it into data tables which can be used for further processing or analysis. It also supports to export the data tables in Microsoft Excel, R, Tabula etc.

OpenRefine

OpenRefine, previously known as GoogleRefine. It is a Java based open source powerful tool for manipulates the huge data. It is used for data loading, understanding, cleaning and transforming from one format to another format. It also supports to extending the data with web services .

CSVKit

CSVKit is a suite for command line tools for converting and working with CSV file. It supports the covert the data from Excel to CSV, JSON to CSV, query with SQL etc.

5.3 COMBINING DATASET

Combining is the process to put two or more dataset together for further processing.

The **pandas** library of Python provides easy functionality to combining the dataset together. Here we learn the combining dataset with **concat**, **merge** and **join** functions using **pandas** library.

- **Concat**: The concat() function is used for combining dataset across the rows or columns.
- **Merge**: The merge() function is used for combining the dataset on common columns.
- **Join**: The join() function is used for combining the dataset on key column or an index.

5.4 CONCATENATING DATASET

The “concat” function is used to perform the concatenation operation with the data frame along with an axis. The datasets are just stitched together along with axis (rows axis and column axis). Here we concatenate the datasets using **pandas**.

Syntax:

pd.concat(objs, axis, join, join_axes, ignore_index, keys)

objs: This is a sequence or mapping of Series, DataFrame, objects.

axis: This is an axis to concatenate. This value is {0, 1, ...}, default is 0.

join: This is used how to handle indexes on another axis(es). This value is {“inner", “outer"}, default is “outer". The outer is used for union operation and inner is used for intersection operation.

join_axes: This is the list of index objects. Its specific indexes to use for the other (n-1) axes instead of performing inner/outer set logic.

ignore_index: This value is Boolean type, default is False. If this value is True, do not use the index values on the concatenation axis. The resulting axis will be labeled 0, ..., n-1.

keys: This is a sequence to add an identifier to the result indexes, default is None.

Example:

Perform the concatenation operation using two different data frames **df1** and **df2**. The below code creates the two different data frame **df1** and **df2**.

```
import pandas as pd

# First dataframe creation

df1 = pd.DataFrame({

        "Name":["Rahul","Shreya","Pankaj","Monika","Kalpesh"],

        "Age":[20, 19, 24, 25, 25],

         "Gender":["Male","Female","Male","Male","Female"],

        "Course":["B.E.","B.Tech.","MCA","M.Tech.","M.E."]}, index = [1, 2, 3, 4, 5])

# Second dataframe creation

df2 = pd.DataFrame({

"Name":["Mayank","Jalpa","Sanjana","Vimal","Raj"],
```

```
"Age":[22, 21, 24, 26, 23],

"Gender":["Male","Female","Female","Male","Male"],
"Course":["MBA","MCA","B.E.","B.Tech.","M.Sc."]},

index = [1, 2, 3, 4, 5])

# Display first dataframe

df1
```

Output:

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.

```
# Display second dataframe

df2
```

	Name	Age	Gender	Course
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

```
# Concatenation of both dataframes
```

```
df3 = pd.concat([df1, df2])

df3
```

Output:

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

```
# Concatenation of both dataframe with axis as an argument

df3 = pd.concat([df1, df2],axis=1)

print(df3)
```

Output:

SN	Name	Age	Gender	Course	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.	Mayank	22	Male	MBA
2	Shreya	19	Female	B.Tech.	Jalpa	21	Female	MCA
3	Pankaj	24	Male	MCA	Sanjana	24	Female	B.E.
4	Monika	25	Male	M.Tech.	Vimal	26	Male	B.Tech.
5	Kalpesh	25	Female	M.E.	Raj	23	Male	M.Sc.

Concatenation of both dataframe with keys as an argument

```
df3 = pd.concat([df1, df2], keys=['x','y'])
```

df3

		Name	Age	Gender	Course
x	1	Rahul	20	Male	B.E.
	2	Shreya	19	Female	B.Tech.
	3	Pankaj	24	Male	MCA
	4	Monika	25	Male	M.Tech.
	5	Kalpesh	25	Female	M.E.
y	1	Mayank	22	Male	MBA
	2	Jalpa	21	Female	MCA
	3	Sanjana	24	Female	B.E.
	4	Vimal	26	Male	B.Tech.
	5	Raj	23	Male	M.Sc.

Concatenation of both dataframe using keys argument

```
df3 = pd.concat([df1, df2], keys=['x','y'], ignore_index=True)
```

df3

	Name	Age	Gender	Course
0	Rahul	20	Male	B.E.
1	Shreya	19	Female	B.Tech.
2	Pankaj	24	Male	MCA
3	Monika	25	Male	M.Tech.
4	Kalpesh	25	Female	M.E.
5	Mayank	22	Male	MBA
6	Jalpa	21	Female	MCA
7	Sanjana	24	Female	B.E.
8	Vimal	26	Male	B.Tech.
9	Raj	23	Male	M.Sc.

Concatenation of both dataframe using append

df1.append(df2)

	Name	Age	Gender	Course
1	Rahul	20	Male	B.E.
2	Shreya	19	Female	B.Tech.
3	Pankaj	24	Male	MCA
4	Monika	25	Male	M.Tech.
5	Kalpesh	25	Female	M.E.
1	Mayank	22	Male	MBA
2	Jalpa	21	Female	MCA
3	Sanjana	24	Female	B.E.
4	Vimal	26	Male	B.Tech.
5	Raj	23	Male	M.Sc.

5.5 MERGING DATASET

The word “merge” and “join” both are used relatively interchangeable in SQL, R and Pandas.

Both merge and join doing the similar things, but there are separate “merge” and “join” functions in Pandas. The merging/joining is the process of bringing two or more datasets together into single dataset and aligning the rows from each dataset based on the common attributes or columns. The “merge‟ function is used to perform the merging operation with the data frame. Here we merge the datasets using pandas.

Syntax:

pd.merge(left, right, how, on, left_on, right_on, left_index, right_index, sort)

left: This is the first data frame.

right: This is the second data frame.

how: This is the method how to perform the merge operation. The values of this field are one of „left‟, „right‟, „inner‟, „outer‟, default is „inner‟.

On: This is the name of column in which action to be perform. This column must be available in both left and right data frame object.

left_on: This is the name of column from left data frame to use as keys.

right_on: This is the name of column from right data frame to use as keys.

left_index: This is using the index (row label) from left data frame as its join keys, if it is True.

right_index: This is used the index (row label) from right data frame as its join keys, if it is True.

sort: This is use to sort the result data frame by join keys in specific order. The value of this field is Boolean, default is True.

Example: Perform the merge operation using two different data frames i.e. left and right.

```
import pandas as pd

# Left dataframe creation

left = pd.DataFrame({

        "Rno":[1, 2, 3, 4, 5],

        "Name":["Rahul","Shreya","Pankaj","Monika","Kalpesh"],
        "Course":["B.E.","B.Tech.","MCA","M.Tech.","M.E."]})

# Right dataframe creation

        right = pd.DataFrame({

                "Rno":[1, 2, 3, 4, 5],

                "Name":["Mayank","Jalpa","Sanjana","Vimal","Raj"],
                "Course":["B.E.","MBA","MCA","B.Tech.","M.Sc."]})

# Display left dataframe

print(left)
```

	Rno	Name	Course
0	1	Rahul	B.E.
1	2	Shreya	B.Tech.
2	3	Pankaj	MCA
3	4	Monika	M.Tech.
4	5	Kalpesh	M.E.

Display right dataframe

print(right)

	Rno	Name	Course
0	1	Mayank	B.E.
1	2	Jalpa	MBA
2	3	Sanjana	MCA
3	4	Vimal	B.Tech.
4	5	Raj	M.Sc.

perform the merge operation on both data frame using ***on*** as an argument.

Merge both left and right dataframe using single on key

pd.merge(left, right, on='Rno')

	Rno	Name_x	Course_x	Name_y	Course_y
0	1	Rahul	B.E.	Mayank	B.E.
1	2	Shreya	B.Tech.	Jalpa	MBA
2	3	Pankaj	MCA	Sanjana	MCA
3	4	Monika	M.Tech.	Vimal	B.Tech.
4	5	Kalpesh	M.E.	Raj	M.Sc.

Merge left and right dataframe using multiple on keys

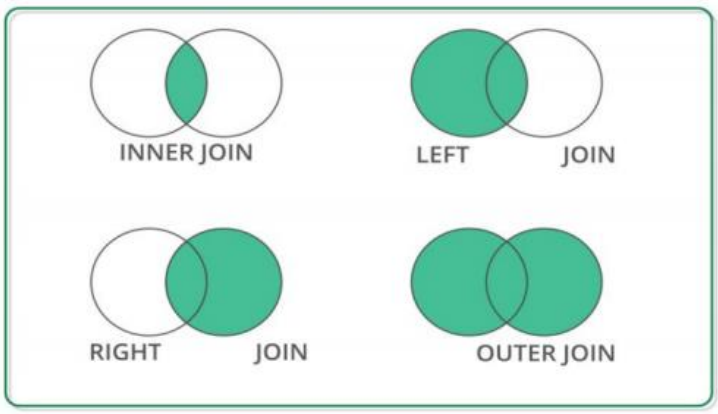
pd.merge(left, right, on=['Rno','Course'])

	Rno	Name_x	Course	Name_y
0	1	Rahul	B.E.	Mayank
1	3	Pankaj	MCA	Sanjana

The **merge** methods are same as SQL **join** equivalent as below:

Merge Method	SQL Join Equivalent	Description
left	Left Outer Join	Use keys from left object
right	Right Outer Join	Use keys from right object
inner	Inner Join	Use intersection of keys
outer	Full Outer Join	Use unions of keys

represented graphically as:



Merge both left and right dataframe using on and how

pd.merge(left, right, on='Course', how='left')

	Rno_x	Name_x	Course	Rno_y	Name_y
0	1	Rahul	B.E.	1.0	Mayank
1	2	Shreya	B.Tech.	4.0	Vimal
2	3	Pankaj	MCA	3.0	Sanjana
3	4	Monika	M.Tech.	NaN	NaN
4	5	Kalpesh	M.E.	NaN	NaN

Merge both left and right dataframe using on and how

pd.merge(left, right, on='Course', how='right')

	Rno_x	Name_x	Course	Rno_y	Name_y
0	1.0	Rahul	B.E.	1	Mayank
1	NaN	NaN	MBA	2	Jalpa
2	3.0	Pankaj	MCA	3	Sanjana
3	2.0	Shreya	B.Tech.	4	Vimal
4	NaN	NaN	M.Sc.	5	Raj

Merge both left and right dataframe using on and how

pd.merge(left, right, on='Course', how='inner')

	Rno_x	Name_x	Course	Rno_y	Name_y
0	1	Rahul	B.E.	1	Mayank
1	2	Shreya	B.Tech.	4	Vimal
2	3	Pankaj	MCA	3	Sanjana

Merge both left and right dataframe using on and how

pd.merge(left, right, on='Course', how='outer')

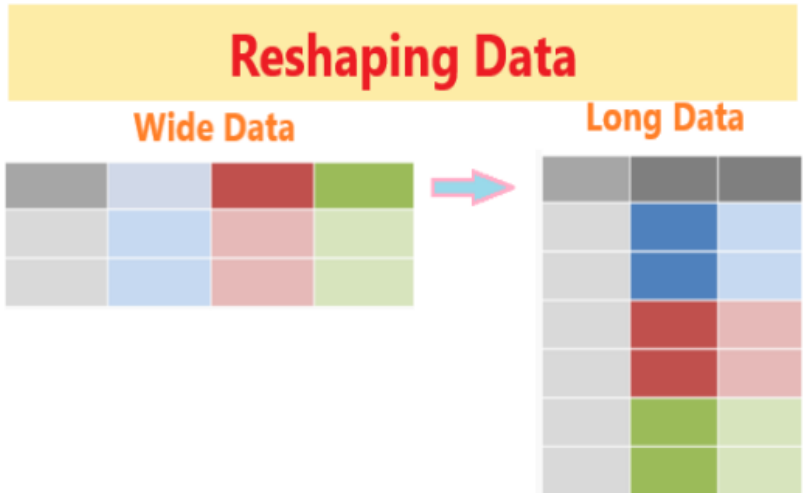
	Rno_x	Name_x	Course	Rno_y	Name_y
0	1.0	Rahul	B.E.	1.0	Mayank
1	2.0	Shreya	B.Tech.	4.0	Vimal
2	3.0	Pankaj	MCA	3.0	Sanjana
3	4.0	Monika	M.Tech.	NaN	NaN
4	5.0	Kalpesh	M.E.	NaN	NaN
5	NaN	NaN	MBA	2.0	Jalpa
6	NaN	NaN	M.Sc.	5.0	Raj

5.6 RESHAPING DATASET

The **dataset** is arranged into rows and columns is referred as the shape of data. In a dataset, each row represents one observation in a vertical or long data and each column is considered a variable with multiple distinct values.

It is needed to convert or transform the dataset from one format to another format, which is called reshaping of dataset.

Reshaping is the process to change the shape or structure of datasets, such as convert ***"wide"*** data tables into ***"long".*** The below figure shows the reshaping process graphically.



The data frame will be reshaped by using **melt**(), **stack**(), **unstack**() and **pivot**() functions as follows:

```
import pandas as pd

# Dataframe creation

df = pd.DataFrame({

"Rno":[1, 2, 3, 4, 5],

"Name":["Rajan","Shital","Mayur","Mittal","Mahesh"],

"Age": [25, 27, 24, 25, 21]})

# Display dataframe

print(df)
```

	Rno	Name	Age
0	1	Rajan	25
1	2	Shital	27
2	3	Mayur	24
3	4	Mittal	25
4	5	Mahesh	21

We can reshape the data frame using **melt**() function. This function is used to wide data frame columns into rows.

```
df.melt()
```

	Variable	value
0	Rno	1
1	Rno	2
2	Rno	3
3	Rno	4
4	Rno	5
5	Name	Rajan
6	Name	Shital
7	Name	Mayur
8	Name	Mittal
9	Name	Mahesh
10	Age	25
11	Age	27
12	Age	24
13	Age	25
14	Age	21

We can reshape the data frame using **stack**() and **unstack**() functions. The stack() function is used to increase the level of index in a data frame.

df.stack()

0	Rno	1
	Name	Rajan
	Age	25
1	Rno	2
	Name	Shital
	Age	27
2	Rno	3
	Name	Mayur
	Age	24

3	Rno	4
	Name	Mittal
	Age	25
4	Rno	5
	Name	Mahesh
	Age	21
dtype: object		

The **unstack**() function is used to do the revert back changes in a data frame was perform by the stack() function.

Rno	0	1
	1	2
	2	3
	3	4
	4	5
Name	0	Rajan
	1	Shital
	2	Mayur
	3	Mittal
	4	Mahesh
Age	0	25
	1	27
	2	24
	3	25
	4	21
dtype: object		

We can reshape the data frame using **pivot**() function. This function is used to reshape the data frame based on the specified column in a data frame.

Performing pivot function on dataframe

df.pivot(columns='Rno')

	Name					**Age**				
Rno	**1**	**2**	**3**	**4**	**5**	**1**	**2**	**3**	**4**	**5**
0	Rajan	NaN	NaN	NaN	NaN	25.0	NaN	NaN	NaN	NaN
1	NaN	Shital	NaN	NaN	NaN	NaN	27.0	NaN	NaN	NaN
2	NaN	NaN	Mayur	NaN	NaN	NaN	NaN	24.0	NaN	NaN
3	NaN	NaN	NaN	Mittal	NaN	NaN	NaN	NaN	25.0	NaN
4	NaN	NaN	NaN	NaN	Mahesh	NaN	NaN	NaN	NaN	21.0

5.7 DATA TRANSFORMATION

Data is collected from the various sources and combine it into a unified data frame. This data frame has large number of columns with different data types.

Data transformation is the process to transform or convert the data as per required format for further processing as and when needed.

The data transformation includes add new columns, find NaN values, drop NaN, replace with mean value, field encoding and decoding, column splitting etc.

5.7.1 Data Frame Creation

To perform the various transformation operation on data, first we have to create the data frame.

The below code will create and display a data frame **df** which contains the three

columns such as **“Name”, “Gender”** and **“City”**.

```
import pandas as pd

# Dataframe creation

df = pd.DataFrame({

"Name":["Jayesh Patel","Priya Shah","Vijay Sharma"],

"Gender": ["Male","Female","Male"],

"City": ["Rajkot","Delhi","Mumbai"]})
```

Display dataframe

print(df)

	Name	Gender	City
0	Jayesh Patel	Male	Rajkot
1	Priya Shah	Female	Delhi
2	Vijay Sharma	Male	Mumbai

5.7.2 Missing Value

The dataset contains the many rows and columns. There are some cells in a dataset that have **NA** or empty cell. This is called **missing data** in a dataset. It is needed first to check that missing value before further processing. The common and very simple method to handle this missing value is to delete the rows which contain missing values.

df.isna().sum()

Here there are no any missing values in each column so it returns zero value in each column. If there are any missing values in each column, it returns the number of missing values.

Name 0

Gender 0

City 0

dtype: int64

drop all the rows which containing missing value.

df = df.dropna()

drop the columns where all elements are missing values.

df.dropna(axis=1, how='all')

drop the columns where any of the elements containing missing values

df.dropna(axis=1, how='any')

keep only the rows which contains maximum two missing values.:

df.dropna(thresh=2)

fill all missing values with mean value of the particular column.

df.fillna(df.mean())

To fill any missing values in specified column with **median** value of the particular column. Here we taken “Age” column for example.

df['Age'].fillna(df['Age'].median())

To fill any missing values in specified column with mode value of the particular column. Here we taken “Age” column for example.

df['Age'].fillna(df['Age'].mode())

5.7.3 Encoding

The dataset contains both numerical and categorical value. The categorical data is not much useful for data processing or analytics. It is needed to encoding the categorical value into numeric value.

Data encoding is the process to convert a categorical variable into a numerical form.

Here we discuss the label encoding which is simply converting each value in a column to a number. Our dataset has **“Gender”** column which has only two values **“male”** and **“female”**

It encodes like this:

Male→0

Female→1

```
df=df.Gender.replace({"Male":0,"Female":1})

print(df)
```

output:

```
0   0
1   1
2   0
Name: Gender, dtype: int64
```

5.7.4 Inset New Column

It is needed to add one or more columns in an existing dataset.

```
df.insert(1, "Age", [21, 23, 24], True)

print(df)
```

	Name	Age	Gender	City
0	Jayesh Patel	21	Male	Rajkot
1	Priya Shah	23	Female	Delhi
2	Vijay Sharma	24	Male	Mumbai

5.7.5 Split Column

To split one column into two or more different columns. The process to create two or more different columns from single column in a dataset is called column splitting. Sometimes the **"Full Name"** column of dataset may be need to split into **"First Name"** and **"Last Name"** as a separate column.

df[['First Name','Last Name']] = df.Name.str.split(expand=True)

df

	Name	Age	Gender	City	First Name	Last Name
0	Jayesh Patel	21	Male	Rajkot	Jayesh	Patel
1	Priya Shah	23	Female	Delhi	Priya	Shah
2	Vijay Sharma	24	Male	Mumbai	Vijay	Sharma

5.8 STRING MANIPULATION

String manipulation is the process of handling and analyzing the strings. The various operation can be performed on string such as string modification, parsing of string, string conversion etc. The various in-built functions available for string manipulation in different language. He we perform some common string manipulation operations in Python.

We take the following strings as an example.

str1 = "Data Science"

str2 = "using"

str3 = "Python"

str4 = "2021"

str5 = " Data Science "

Function	Description	Example	Output
capitalize()	Converts the first character of string into upper case	str2.capitalize()	‘Using’
casefold()	Converts the string into lower case	str1.casefole()	‘data science’
center()	Returns the string in center of the specified size	str1.center(15)	‘ Data Science’
count()	Returns the number of times a specified value occurs in a string	str1.count(“a”)	2
endswith()	Returns true if the string ends with the specified value	str1.endswith(“nce”)	True
find()	Searches the string for a specified value and returns the	str1.find(“i”)	7

	position of where it is found.		
format()	Formats the specified values in a string	str4.format()	'2021'
index()	Searches the string for a specified value and returns the position of where it was found	str1.index("c")	6
isalnum()	returnsTrue if all the characters in a string are alphanumeric	str4.isalnum()	True
isalpha()	Returns True if all characters in a string are alphabet	str2.isalpha()	True
isdigit()	Returns True if all characters in a string are digits	str4.isdigit()	True
islower()	Returns True if all characters in a string are lower case	str2.islower()	True
isnumeric()	Returns True if all characters in a string are numeric	str4.isnumeric()	True

isprintable()	Returns True if all characters in a string are printable	str1.isprintable()	True
isspace()	Returns True if all characters in a string are whitespaces	str1.isspace()	False
istitle()	Returns True if the string follows the title case rules	str1.istitle()	True
isupper()	Returns True if all characters in a string are upper case letter	str1.isupper()	False
len(string)	Returns the length of a string	len(str1)	12
lower()	Converts a string into lower case	str1.lower()	‘data science’
lstrip()	Returns the string with left trim version	str5.lstrip()	‘Data Science ‘
replace(old,new)	Replaces the old string with new string	Str1.replace(“Science”, “Analytics”)	“Data Analytics”
rfind()	Searches the string for a specified value and returns the last position of where it was found	str1.rfind(“e”)	11

rindex()	Searches the string for a specified value and returns the last index position of where it is found	str1.rindex(“a”)	3
rstrip()	Returns the string with right trim version	str1.rstrip(5)	‘ Data Science’
split()	Splits the string with specified separator and returns a list	str1.split(“ “)	[‘Data’, ‘Science’]
startswith()	Returns true if the string starts with the specified value	str3.startswith(“P”)	True
strip()	Returns both left and right trim version	str5.strip()	‘Data Science’
swapcase()	Returns the swaps cases, lower case becomes upper case and vice versa	str1.swapcase()	‘dATA sCIENCE’
title()	Converts the first character of each word to upper case	str2.title()	‘Using’
upper()	Converts a string into upper case	str1.upper()	‘DATA SCIENCE’

zfill()	Returns the string with filled by 0 for specified number of times in a string at beginning	str3.zfill(10)	'0000Python'
+	Concatenates two strings	str1+" "+str2+" "+str3	"Data Science using Python"
*	Repeat the string n times	str3*3	"PythonPythonPython"
string[0]	Returns the first character of a string	str1[0]	'D'
string[7]	Returns the eighth character of a string	str1[7]	'i'
string[2:8]	Returns the third to eighth characters	str1[2:8]	'ta Sci'
string[3:]	Returns characters from fourth to last	str1[3:]	's Science'
string[:8]	Returns characters from fourth to last	str1[:8]	'Data Sci'
string[-4:]	Returns the last four characters	str1[-4:]	'ence'
string[:-4]	Removes the last four characters	str1[:-4]	'Data Sci'

5.9 REGULAR EXPRESSION

Regular expression or RegEx is generally used to identify whether a sequence or character or pattern is exists in a given string or not. It is also used to identify the position of such pattern in a string or file. It mainly used to find and replace specific patterns in a string or file. It helps to manipulate text-based datasets.

Python has a built-in package called ***re***, to work with Regular Expression.

The ***re*** package of Python has set of functions to search the string for matching. The common Python RegEx functions are as follow:

Function	Description
findall	Returns a list of all match values
search	Returns a match object if math found anywhere in the string
split	Returns a list and split the string where each match found
sub	Replaces the one or more matches with a specified string

5.9.1 The findall() Function

This function returns a list which contains all match values.

Example:

```
import re

string = "Working with Data Science using Python"

result = re.findall("th",string)
```

```
print(result)
```

The list contains all match values in an order of they are found.

Output:

```
['th', 'th']
```

Example:

```
import re
string = "Working with Data Science using Python"
result = re.findall("ds",string)
print(result)
```

If no matches found, an empty list will return.

Output:

```
[]
```

5.9.2 The search() Function

This function returns a match object if match found anywhere in the string. Only first occurrence of match will be returned, if there are more than one match found. The none will return if no match found.

Example:

```
import re
```

```
string = "Working with Data Science using Python"

result = re.search("wi",string)

result
```

It found the match value "***wi***".

Output:

```
<re.Match object; span=(8, 10), match='wi'>
```

Example:

```
import re

string = "Working with Data Science using Python"

result = re.search("\s",string)

result
```

It found the match value "**\s**" i.e. space.

Output:

```
<re.Match object; span=(7, 8), match=' '>
```

5.9.3 The split() Function

This function returns a list of where the string has been split at each match found.

Example:

```
import re
```

```
string = "Working with Data Science using Python"

result = re.split("\s",string)

result
```

It found the match value **“\s”** i.e. space.

Output:

```
['Working', 'with', 'Data', 'Science', 'using', 'Python']
```

Example:

```
import re

string = "Working with Data Science using Python"

result = re.split("\s",string,2)

result
```

It found the match value **“\s”** i.e. space. But here the string will split into first 2 occurrence of found only. The remaining string will print as it is.

Output:

```
['Working', 'with', 'Data Science using Python']
```

5.9.4 The sub() Function

This function replaces the string with the specified text at each match found.

It will print same string, if not match found.

Example:

```
import re
string = "Working with Data Science using Python"
result = re.sub("\s", "-",string)
result
```

It found the match value **“\s”** i.e. space. Every **“\s”** (space) will replace with the **“-”** (dash).

Output:

'Working-with-Data-Science-using-Python'

Example:

```
import re
string = "Working with Data Science using Python"
result = re.sub("\s","-",string,2)
result
```

It found the match value **“\s”** i.e. space. But here **“\s”** (space) will replace with the **“-”** (dash) in first two occurrence of found only. The remaining string will print as it is.

Output:

'Working-with-Data Science using Python’

QUESTIONS

1. What is data wrangling?

2. What is data cleaning?

3. List tools for data wrangling.

4. What is merging dataset?

5. What is reshaping dataset?

6. Explain steps for data wrangling process.

7. Explain concat function with example.

8. Explain merge operation with syntax and example.

9. Explain reshaping dataset different functions.

10. Explain string function with example.

11. Explain regular expression with example.

12. Create and display a data frame.

13. Create a data frame and find null values and remove it.

14. Create a data frame and inert new column in data frame.

15. Create a data frame and convert categorical data into numerical values.

16. Create a data frame and split the column.

17. Create data frames and combine using concat function.

18. Create data frames and merge with different arguments.

19. Create data frame and reshape using melt function.

20. Perform string manipulation operations on string.

21. Perform regular expression functions on string.

Statistics for Data Science

- a **population** is a collection of objects, items (“units”) about which information is sought;

- a **sample** is a part of the population that is observed

Variables

A variable is an attribute of an object of study that may vary for different cases. Thus, a variable varies for different case studies in research. Considering the previous example of the survey on the temperature of many cities worldwide on the same day, the variables are "Temperature" and "City" because both the attributes vary for different cases.

Now variables can be of two types.

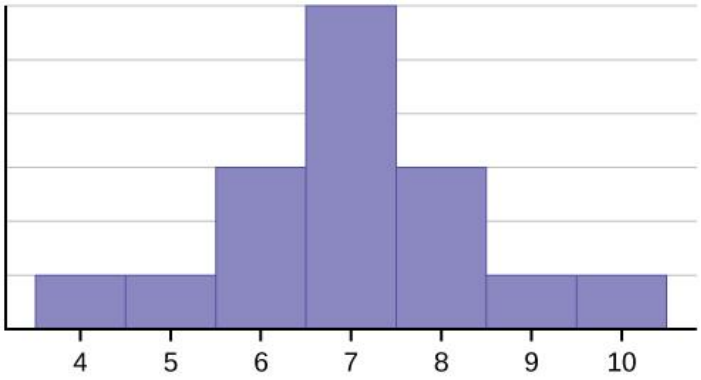
1. **Numerical variable**: They represent values that have numbers. For Example, age, weight, height.

2. Categorical variable

These variables represent values that have words, for example, name, nationality, sport, etc. For our previous example of the survey, "Temperature" is a numerical variable.

Consider the following data set.
4; 5; 6; 6; 6; 7; 7; 7; 7; 7; 7; 8; 8; 8; 9; 10

This data set can be represented by following histogram. Each interval has width one, and each value is located in the middle of an interval.



Statistics:

Descriptive statistics helps to simplify large amounts of data in a sensible way. In contrast to inferential statistics, in descriptive statistics we do not draw conclusions beyond the data we are analyzing; neither do we reach any conclusions regarding hypotheses we may make. We do not try to infer characteristics of the “population” of the data, but claim to present quantitative descriptions of it in a manageable form. It is simply a way to describe the data.

inferential statistics

Descriptive statistics

1- mean

One of the first measurements we use to have a look at the data is to obtain sample statistics from the data, such as the sample mean. Given a sample of n values, {xi},i = 1,..., n, the mean, **μ**, is the sum of the values divided by the number of values,

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i .$$

```
import pandas as pd

 # list of name, degree, score

nme = ["aparna", "pankaj", "sudhir", "Geeku"]

deg = ["MBA", "BCA", "M.Tech", "MBA"]

scr = [90, 40, 80, 98]


# dictionary of lists

dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)

print(df)

print()

print()

fm['score'].mean()
```

```
In [56]: import pandas as pd
          # list of name, degree, score
         nme = ["aparna", "pankaj", "sudhir", "Geeku"]
         deg = ["MBA", "BCA", "M.Tech", "MBA"]
         scr = [90, 40, 80, 98]

         # dictionary of lists
         dict = {'name': nme, 'degree': deg, 'score': scr}

         df = pd.DataFrame(dict)
         print(df)
         print()
         print()
         fm['score'].mean()

             name  degree  score
         0  aparna     MBA     90
         1  pankaj     BCA     40
         2  sudhir  M.Tech     80
         3   Geeku     MBA     98


Out[56]: 40.0
```

2- Sample variance

The mean is not usually a sufficient descriptor of the data. We can go further by knowing two numbers: mean and variance.

$$\sigma^2 = \frac{1}{n}\sum_i (x_i - \mu)^2.$$

print(fm['score'].var())

3- Sample median

The **mean** of the samples is a good descriptor, but it has an important drawback: what will happen if in the sample set there is an error with a value very different from the rest?

For example, considering hours worked per week, it would normally be in a range between 20 and 80; but what would happen if by mistake there was a value of 1000? An item of data that is significantly different from the rest of the data is called an **outlier**. In this case, the mean, μ, will be drastically changed towards the **outlier**. One solution to this drawback is offered by the statistical median, μ12, which is an order statistic giving the middle value of a sample

print(fm['score'].median())

4- Quantiles and Percentiles

we can order the samples {xi}, then find the x p so that it divides the data into two parts, where

• a fraction p of the data values is less than or equal to x_p and

• the remaining fraction (1 − p) is greater than x_p.

That value, x_p, is the p-th quantile, or the 100 × p-th percentile.

For example, a 5-number summary is defined by the values x_{min}, Q1, Q2, Q3, x_{max} , where Q1 is the 25 × p-th percentile, Q2 is the 50 × p-th percentile and Q3 is the 75 × p-th percentile.

5- Measuring Asymmetry: Skewness and Pearson's Median Skewness Coefficient

For univariate أحادي المتغير data, the formula for **skewness** is a statistic that measures the asymmetry of the set of n data samples, x_i :

$$g_1 = \frac{1}{n}\frac{\sum_i (x_i - \mu^3)}{\sigma^3},$$

where **μ** is the mean, **σ** is the standard deviation, and **n** is the number of data points. Negative deviation indicates that the distribution "skews left" (it extends further to the left than to the right).

Note: the skewness for a **normal distribution** is **zero**, and any **symmetric** data must have a skewness of zero.

Type of Skewness

Various types of skewness used in mathematics are,

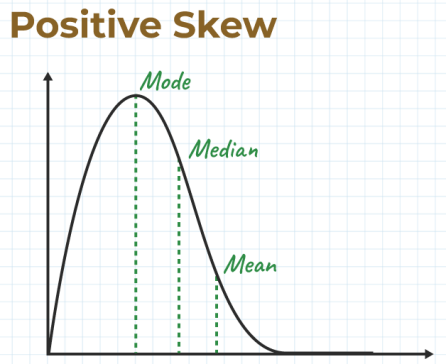
- Positive Skewness
- Negative Skewness
- Zero Skewness

Positive Skewness

Positive Skewness means the **tail** on the **right** side of the distribution is **longer**.

The mean and median will be greater than the mode.

Condition for positive skewness = **Mean > Median >Mode**



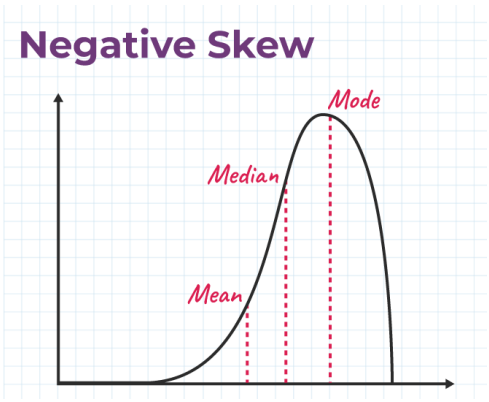
Let's take an example of the income distribution where a **few** people earn **very** high incomes and the majority earn lower incomes. so, this is often **positively** skewed. Analyzing skewed data can provide valuable insights into the underlying causes and potential solutions or interventions.

Negative Skewness

Negative Skewness means when the **tail** of the **left** side of the distribution is **longer** than the tail on the right side.

The **mean** and **median** will be **less** than the **mode**.

Condition for negative skewness is **Mode > Median > Mean**
The curve shows negative skewness in the image below,



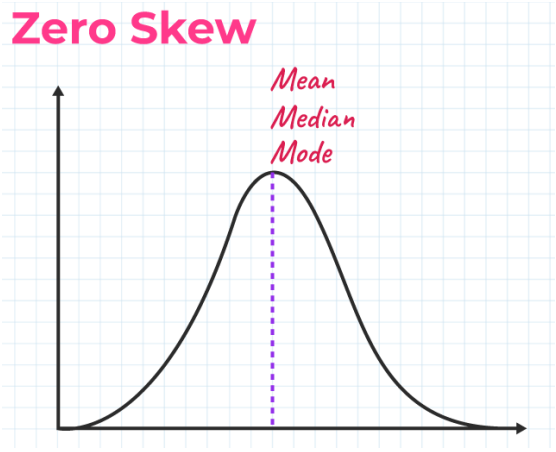
Let's take an example of a match, during the match most of the players of a particular team scored runs above 50 and only a few of them scored below 10. In such a case, the data is generally represented with the help of a negatively skewed distribution. And this data is helpful to analyze the game's performance.

Zero Skewness

It is also known as a "symmetric distribution".

It signifies that distribution of data is evenly distributed around the **mean**, with no long tails on either end of the distribution.

Condition for zero skewness is **Mean = Mode = Median**
The curve for zero skews is shown in the image below,



The **Pearson's median skewness coefficient** is a more robust alternative to the skewness coefficient and is defined as follows:

$$g_p = 3(\mu - \mu_{12})\sigma$$

μ mean, μ_{12} median

```
def pearson(x):
    return 3*(x.mean() - x.median())*x.std()
print "Pearson's coefficient of x = ", pearson(x)
```

Covariance, and Pearson's and Spearman's Rank Correlation

Variables of data can express relations. For example, countries that tend to invest in research also tend to invest more in education and health. This kind of relationship is captured by the covariance.

يمكن لمتغيرات البيانات التعبير عن العلاقات. على سبيل المثال، تميل البلدان التي تميل إلى الاستثمار في البحوث إلى زيادة الاستثمار في التعليم والصحة. يتم التقاط هذا النوع من العلاقات من خلال **التغاير**.

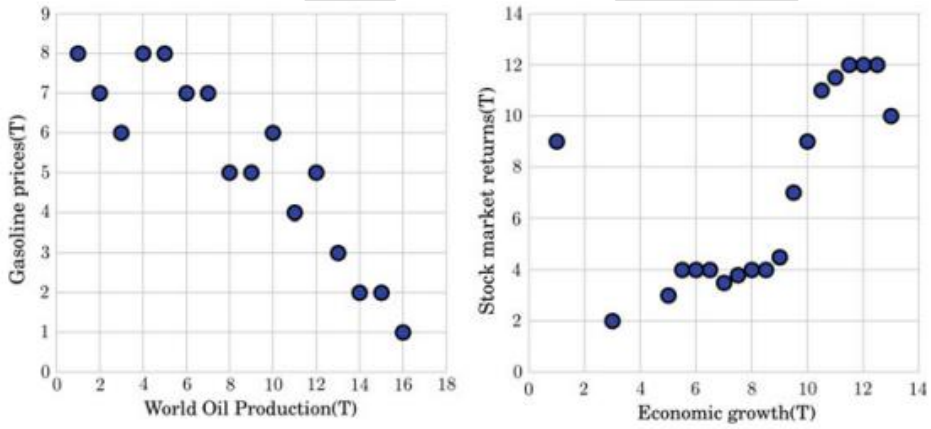


Fig. 3.9 Positive correlation between economic growth and stock market returns worldwide (*left*). Negative correlation between the world oil production and gasoline prices worldwide (*right*)

Covariance is a measure of the relationship between two random variables and to what extent, they change together. Or we can say, in other words, it defines the changes between the two variables, such that change in one variable is equal to change in another variable. This is the property of a function of maintaining its form when the variables are linearly transformed. Covariance is measured in units, which are calculated by multiplying the units of the two variables.

Population Covariance Formula

$$Cov(x,y) = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{N}$$

Sample Covariance

$$Cov(x,y) = \frac{\sum(x_i - \bar{x})(y_i - y)}{N - 1}$$

Types of Covariance

Covariance can have both **positive** and **negative** values. Based on this, it has two types:

1- Positive Covariance
2- Negative Covariance

Positive Covariance

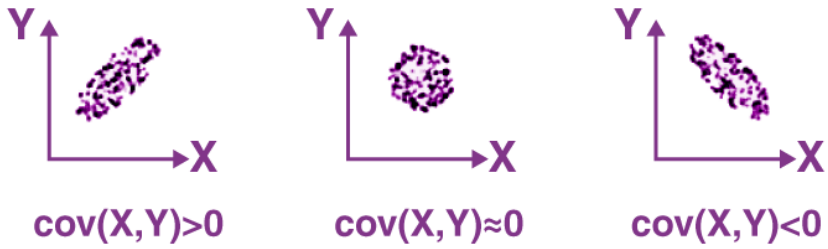
If the covariance for any two variables is positive, that means, both the variables move in the same direction.

Here, the variables show similar behavior.

That means, if the values (greater or lesser) of one variable corresponds to the values of another variable, then they are said to be in positive covariance.

Negative Covariance

If the covariance for any two variables is negative, that means, both the variables move in the opposite direction. It is the opposite case of positive covariance, where greater values of one variable correspond to lesser values of another variable and vice-versa.



If cov(X, Y) is greater than zero, then we can say that the covariance for any two variables is positive and both the variables move in the same direction.

If cov(X, Y) is less than zero, then we can say that the covariance for any two variables is negative and both the variables move in the opposite direction.

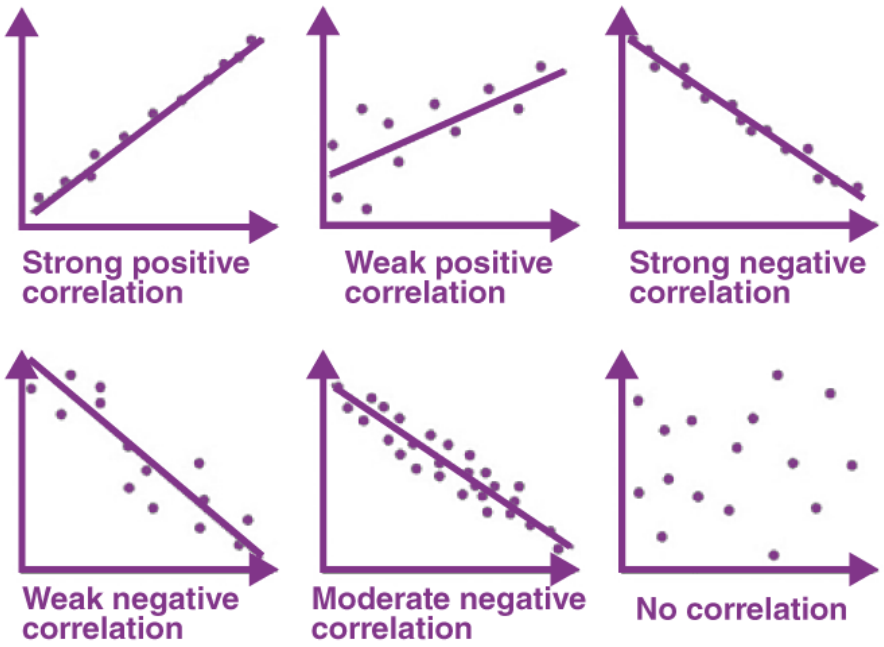
If cov(X, Y) is zero, then we can say that there is no relation between two variables.

Correlation Coefficient Formula

We have already discussed covariance, which is the evaluation of changes between any two variables. Correlation estimates the depth of the relationship between variables. It is the estimated measure of covariance and is dimensionless. In other words, the correlation coefficient is a constant value always and does not have any units. The relationship between the correlation coefficient and covariance is given by;

$$\text{Correlation}, \rho(X,Y) = \text{Cov}(X,Y)/\sigma_x \sigma_y$$

Based on the value of correlation coefficient, we can estimate the type of correlation between the given two variables. Also, the graphical representation of correlation among two variables is given in the below figure.



Note that the **Pearson's correlation** is always between −1 and +1, where the magnitude depends on the degree of correlation.

If the Pearson's correlation is 1 (or −1), it means that the variables are **perfectly** correlated (positively or negatively)

Spearman's Rank Correlation

The Spearman's rank correlation comes as a solution to the robustness problem of Pearson's correlation when the data contain outliers.

The main idea is to use the **ranks** of the **sorted** sample data, instead of the values themselves.

For example, in the list [4, 3, 7, 5], the rank of 4 is 2, since it will appear second in the ordered list ([3, 4, 5, 7]).

Spearman's correlation computes the correlation between the ranks of the data.

For example, considering the data: X=[10, 20, 30, 40, 1000], and Y= [−70, −1000, −50, −10, −20], where we have an outlier in each one set.

If we compute the ranks, they are [1.0, 2.0, 3.0, 4.0, 5.0] and [2.0, 1.0, 3.0, 5.0, 4.0].

$$\rho = 1 - \frac{6\sum d_i^2}{n(n^2-1)}$$

As value of the Pearson's coefficient, we get 0.28, which does not show much correlation between the sets. However, the Spearman's rank coefficient, capturing the correlation between the ranks, gives as a final value of 0.80, confirming the correlation between the sets. As an exercise, you can compute the Pearson's and the Spearman's rank correlations for the different Anscombe configurations given in Fig. 3.10. Observe if linear and nonlinear correlations can be captured by the Pearson's and the Spearman's rank correlations.

Example:

consider the following example data regarding the marks achieved in a maths and English exam:

	Marks									
English	56	75	45	71	61	64	58	80	76	61
Maths	66	70	40	60	65	56	59	77	67	63

The procedure for ranking these scores is as follows:

First, create a table with four columns and label them as below:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
56	66	9	4
75	70	3	2
45	40	10	10
71	60	4	7

English (mark)	Maths (mark)	Rank (English)	Rank (maths)
61	65	6.5	5
64	56	5	9
58	59	8	8
80	77	1	1
76	67	2	3
61	63	6.5	6

You need to rank the scores for maths and English separately. The score with the highest value should be labelled "1" and the lowest score should be labelled "10" (if your data set has more than 10 cases then the lowest score will be how many cases you have). Look carefully at the two individuals that scored 61 in the English exam (highlighted in bold). Notice their joint rank of 6.5. This is because when you have two identical values in the data (called a "tie"), you need to take the average of the ranks that they would have otherwise occupied. We do this because, in this example, we have no way of knowing which score should be put in rank 6 and which score should be ranked 7. Therefore, you will notice that the ranks of 6 and 7 do not exist for English. These two ranks have been averaged ((6 + 7)/2 = 6.5) and assigned to each of these "tied" scores.

An **example** of calculating Spearman's correlation

To calculate a Spearman rank-order correlation on data without any ties we will use the following data:

	Marks									
English	56	75	45	71	62	64	58	80	76	61
Maths	66	70	40	60	65	56	59	77	67	63

We then complete the following table:

English (mark)	Maths (mark)	Rank (English)	Rank (maths)	d	d^2
56	66	9	4	5	25
75	70	3	2	1	1
45	40	10	10	0	0
71	60	4	7	3	9
62	65	6	5	1	1
64	56	5	9	4	16
58	59	8	8	0	0
80	77	1	1	0	0
76	67	2	3	1	1
61	63	7	6	1	1

Where d = difference between ranks and d^2 = difference squared.

We then calculate the following:

$$\sum d_i^2 = 25 + 1 + 9 + 1 + 16 + 1 + 1 = 54$$

We then substitute this into the main equation with the other information as follows:

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

$$\rho = 1 - \frac{6 \times 54}{10(10^2 - 1)}$$

$$\rho = 1 - \frac{324}{990}$$

$$\rho = 1 - 0.33$$

$$\rho = 0.67$$

as n = 10. Hence, we have a ρ (or r_s) of 0.67. This indicates a strong positive relationship between the ranks individuals obtained in the maths and English exam.

That is, the higher you ranked in maths, the higher you ranked in English also, and vice versa.

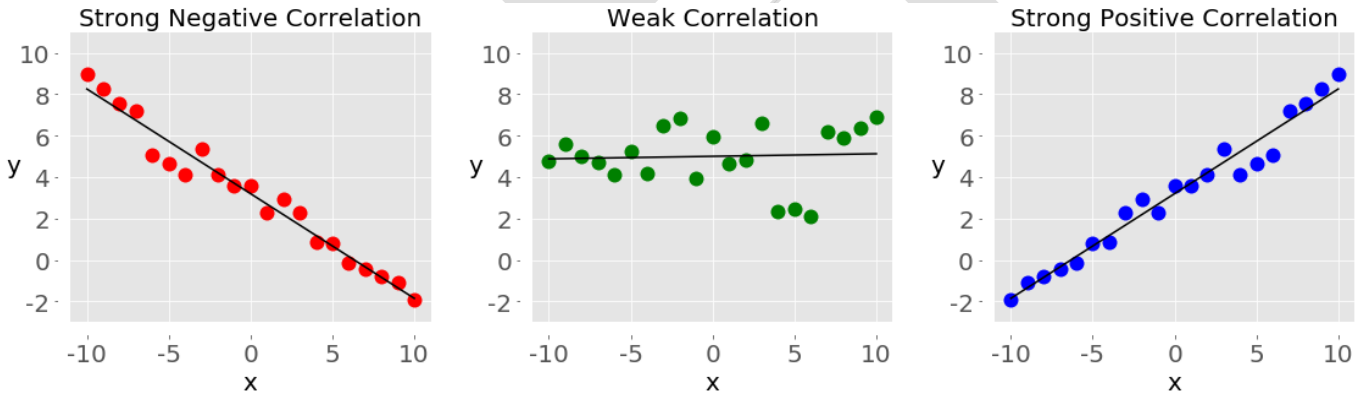
Example:

When data is represented in the form of a table, the rows of that table are usually the observations, while the columns are the features. Take a look at this employee table:

Name	Years of Experience	Annual Salary
Ann	30	120,000
Rob	21	105,000
Tom	19	90,000
Ivy	10	82,000

In this table, each row represents one observation, or the data about one employee (either Ann, Rob, Tom, or Ivy). Each column shows one property or feature (name, experience, or salary) for all the employees.

If you analyze any two features of a dataset, then you'll find some type of **correlation** between those two features. Consider the following figures:

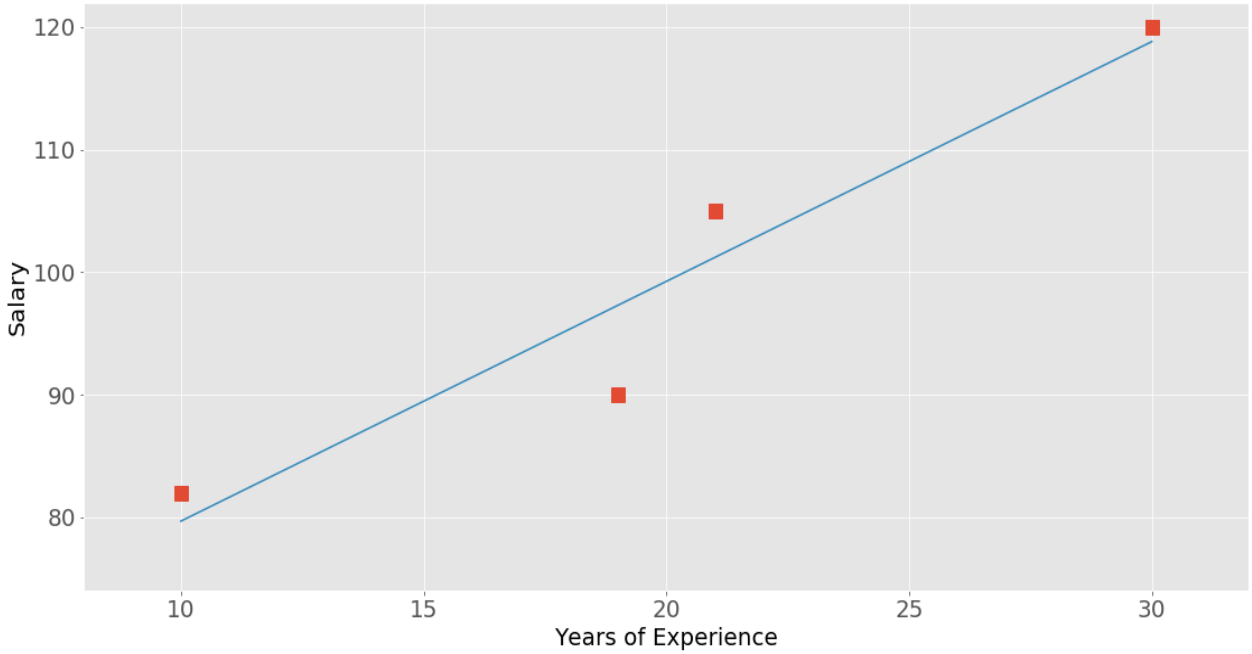


Each of these plots shows one of three different forms of correlation:

1. **Negative correlation (red dots):** In the plot on the left, the y values tend to decrease as the x values increase. This shows strong negative correlation, which occurs when *large* values of one feature correspond to *small* values of the other, and vice versa.
2. **Weak or no correlation (green dots):** The plot in the middle shows no obvious trend. This is a form of weak correlation, which occurs when an association between two features is not obvious or is hardly observable.

3. **Positive correlation (blue dots):** In the plot on the right, the y values tend to increase as the x values increase. This illustrates strong positive correlation, which occurs when *large* values of one feature correspond to *large* values of the other, and vice versa.

The next figure represents the data from the employee table above:



The correlation between experience and salary is positive because higher experience corresponds to a larger salary and vice versa.

```
In [21]: import numpy as np
x = np.arange(10, 20)
print("x=",x)
# array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19])
y = np.array([2, 1, 4, 5, 8, 12, 18, 25, 96, 48])
print("y=",y)
# array([ 2,  1,  4,  5,  8, 12, 18, 25, 96, 48])
r = np.corrcoef(x, y) # Pearson correlation coefficient
print("r=",r)
print("r[0, 1]",r[0, 1])
print("r[1, 0]",r[1, 0])

x= [10 11 12 13 14 15 16 17 18 19]
y= [ 2  1  4  5  8 12 18 25 96 48]
r= [[1.         0.75864029]
 [0.75864029 1.        ]]
r[0, 1] 0.7586402890911867
r[1, 0] 0.7586402890911866

In [ ]:
```

The values on the main diagonal of the correlation matrix (upper left and lower right) are equal to 1. The upper left value corresponds to the correlation coefficient for x and x, while the lower right value is the correlation coefficient for y and y. They are always equal to 1.

Kendal tau Rank Correlation

The Kendall tau-b correlation coefficient, τ_b, is a nonparametric measure of association based on the number of concordances and discordances in paired observations.

Suppose two observations (X_i, Y_i) and (X_j, Y_j) are ***concordant*** if they are in the same order with respect to each variable. That is, if

1. $X_i < X_j$ and $Y_i < Y_j$, or if
2. $X_i > X_j$ and $Y_i > Y_j$

They are ***discordant*** if they are in the reverse ordering for *X* and *Y*, or the values are arranged in opposite directions. That is, if

1. $X_i < X_j$ and $Y_i > Y_j$, or if
2. $X_i > X_j$ and $Y_i < Y_j$

The two observations are ***tied*** if $X_i=X_j$ and/or $Y_i=Y_j$

The total number of pairs that can be constructed for a sample size of ***n*** is

$$N = \binom{n}{2} = \frac{1}{2}n(n-1)$$

N can be decomposed into these five quantities:

$$N = P + Q + X_0 + Y_0 + (XY)_0$$

where P is the number of concordant pairs, *Q* is the number of discordant pairs, X_0 is the number of pairs tied only on the *X* variable, Y_0 is the number of pairs tied only on the *Y* variable, and $(XY)_0$ is the number of pairs tied on both *X* and *Y*.

The Kendall tau-b for measuring order association between variables *X* and *Y* is given by the following formula:

$$t_b = \frac{P - Q}{\sqrt{(P + Q + X_0)(P + Q + Y_0)}}$$

-1:1

Example:

Age and **percentage** body fat were measured in 18 adults. Calculate each of the Pearson, Spearman, and Kendall correlation coefficients. AS:

No.	Age	fat%
01	23	9.5
02	23	27.9
03	27	7.8
04	27	17.8
05	39	31.4
06	41	25.9
07	45	27.4
08	49	25.2
09	50	31.1
10	53	34.7

```
11    53    42.0
12    54    29.1
13    56    32.5
14    57    30.3
15    58    33.0
16     58    33.8
17     60     41.1
18     61     34.5
```

Question:

Calculate the coefficient of covariance for the following data:

X	2	8	18	20	28	30
Y	5	12	18	23	45	50

Solution:

Number of observations = 6

Mean of X = 17.67

Mean of Y = 25.5

Cov(X, Y) = (⅙) [(2 – 17.67)(5 – 25.5) + (8 – 17.67)(12 – 25.5) + (18 – 17.67)(18 – 25.5) + (20 – 17.67)(23 – 25.5) + (28 – 17.67)(45 – 25.5) + (30 – 17.67)(50 – 25.5)]

= 157.83

EXERCISES:

Use NumPy to:

1) Create an arbitrary one-dimensional array called "**v**".

```
import numpy as np

v= np.array([3,8,12,18,7,11,30])
```

2) Create a new array which consists of the odd indices of previously created array "v".

```
odd_elements = v[1::2]
```

3) Create a new array in backwards ordering from v.

```
reverse_order = v[::-1]
```

4) What will be the output of the following code:

```
a = np.array([1, 2, 3, 4, 5])

b = a[1:4]

b[0] = 200

print(a[1])
```

The output will be 200, because slices are views in numpy and not copies.

5) Create a two-dimensional array called "m".

```
m = np.array([ [11, 12, 13, 14], [21, 22, 23, 24], [31, 32, 33, 34]])
```

6) Create a new array from m, in which the elements of each row are in reverse order.

```
m[::,::-1]
```

7) Another one, where the rows are in reverse order.

```
m[::-1]
```

8) Create an array from m, where columns and rows are in reverse order.

```
m[::-1,::-1]
```

9) Cut of the first and last row and the first and last column

```
m[1:-1,1:-1]
```

10) Define an int16 data type and call this type i16. The elements of the list 'lst' are turned into i16 types to create the two-dimensional array A.

```
import numpy as np

i16 = np.dtype(np.int16)

print(i16)

 lst = [ [3.4, 8.7, 9.9], [1.1, -7.8, -0.7], [4.1, 12.3, 4.8] ]

A = np.array(lst, dtype=i16)
```

```
print(A)
```

```
import numpy as np
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])
population_table = np.array([
('Netherlands', 393, 41526, 16928800),
('Belgium', 337, 30510, 11007020),
('United Kingdom', 256, 243610, 62262000),
('Germany', 233, 357021, 81799600),
('Liechtenstein', 205, 160, 32842),
('Italy', 192, 301230, 59715625),
('Switzerland', 177, 41290, 7301994),
('Luxembourg', 173, 2586, 512000),
('France', 111, 547030, 63601002),
('Austria', 97, 83858, 8169929),
('Greece', 81, 131940, 11606813),
   ('Ireland', 65, 70280, 4581269),
('Sweden', 20, 449964, 9515744),
('Finland', 16, 338424, 5410233),
('Norway', 13, 385252, 5033675)],
dtype=dt)
print(population_table[:4])
```

```
In [1]: import numpy as np
dt = np.dtype([('country', 'S20'), ('density', 'i4'), ('area', 'i4'), ('population', 'i4')])
population_table = np.array([
('Netherlands', 393, 41526, 16928800),
('Belgium', 337, 30510, 11007020),
('United Kingdom', 256, 243610, 62262000),
('Germany', 233, 357021, 81799600),
('Liechtenstein', 205, 160, 32842),
('Italy', 192, 301230, 59715625),
('Switzerland', 177, 41290, 7301994),
('Luxembourg', 173, 2586, 512000),
('France', 111, 547030, 63601002),
('Austria', 97, 83858, 8169929),
('Greece', 81, 131940, 11606813),
    ('Ireland', 65, 70280, 4581269),
('Sweden', 20, 449964, 9515744),
('Finland', 16, 338424, 5410233),
('Norway', 13, 385252, 5033675)],
dtype=dt)
print(population_table[:4])
```

```
[(b'Netherlands', 393,  41526, 16928800)
 (b'Belgium', 337,  30510, 11007020)
 (b'United Kingdom', 256, 243610, 62262000)
 (b'Germany', 233, 357021, 81799600)]
```

print(population_table['density'])

print(population_table['country'])

print(population_table['area'][2:5])

```
In [2]: print(population_table['density'])
print(population_table['country'])
print(population_table['area'][2:5])
```

```
[393 337 256 233 205 192 177 173 111  97  81  65  20  16  13]
[b'Netherlands' b'Belgium' b'United Kingdom' b'Germany' b'Liechtenstein'
 b'Italy' b'Switzerland' b'Luxembourg' b'France' b'Austria' b'Greece'
 b'Ireland' b'Sweden' b'Finland' b'Norway']
[243610 357021    160]
```

```
In [ ]:
```

12) Attendance of all classes of a school are as follows find their skewness?
1st (35), 2nd(32), 3rd(38), 4th(39), 5th(43)

lass Name	Number of students
1 st	35
2 nd	32
3 rd	38
4 th	39
5 th	45

Q) What does positive skewness mean?

Answer:

Positive skewness means the distribution is skewed to the right. There are more values on the right side of the mean than on the left side.

Condition for Positive Skewness = Mean > median >mode

Q: What does negative skewness mean?

Answer:

Negative skewness means the distribution is skewed to the left. There are more values on the left side of the mean than on the right side.

Condition for negative skewness = Mode > median > mean

Q: What is the formula for calculating skewness?

$$g_1 = \frac{1}{n}\frac{\sum_i (x_i - \mu^3)}{\sigma^3},$$